

CLOUD COMPUTING
CSE 838

Project Report

Team Members:

Vaibhav Mittal (IMT2022126)
Aaditya Joshi (IMT2022092)

Contents

1	Introduction	2
2	Assumptions	2
3	System Design	2
3.1	Architecture Overview	2
3.2	Data Flow	3
3.3	Failure Handling	3
3.4	API Endpoints	3
3.5	Project Flow	4
3.5.1	System Initialization	4
3.5.2	PUT Operation Flow	4
3.5.3	Read Operation Flow (GET)	5
3.5.4	Heartbeat and Failure Detection	5
3.5.5	Re-Replication Upon Worker Failure	6
3.6	System Flow Diagram	7
4	AI Usage	7
5	Contribution	8

1 Introduction

Github Link: [Project Code](#)

In this project we have built a distributed key value store consisting of a single controller node and 4 individual worker nodes.

We support two primary operations:-

- **Put-** Takes a key and value and associates the key with that value.
- **Get-** Takes a key as input and returns the value

2 Assumptions

During the design and implementation of the distributed key-value store, several assumptions were made to simplify system behavior and focus on the core objectives. These assumptions are as follows:

1. **Fixed Number of Workers:** The number of worker nodes in the system is fixed and cannot increase dynamically. All key-space partitioning and replication strategies are based on this predefined configuration.
2. **Controller Reliability:** The controller node is assumed to be fault-tolerant and cannot fail. All metadata related to key partitioning, worker mapping, and failure detection relies on the controller being consistently available.
3. **GET Request Model:** For every GET operation, the client first queries the controller to retrieve the primary worker responsible for the requested key. Once this mapping is known, the client communicates directly with the worker to obtain the value.
4. **PUT Operation Flow:** For each PUT command, the key-value pair is first sent to the primary worker. The primary worker then synchronously writes the data to one replica to achieve consistency, and subsequently triggers an asynchronous write to the third replica.

3 System Design

The design of the distributed key-value store is centered around scalability, reliability, and fault tolerance while maintaining simplicity in client interaction. The system is composed of one controller node and four worker nodes, where the controller manages metadata and workers handle data storage and replication. The following subsections describe the architecture, data flow, API structure, and failure handling in the system.

3.1 Architecture Overview

The architecture follows a controller-worker model. The controller is responsible for partitioning the key-space, assigning primary workers, maintaining replication metadata, and monitoring worker health through periodic heartbeats. Each worker node is designated as a primary storage node for a subset of keys and also stores replicas from other workers.

Keys are partitioned based on the total number of workers, which is fixed. Each key is stored on three workers: one primary and two replicas. Of these replicas, the first is written synchronously

to ensure durability during a PUT operation, while the second replica is written asynchronously in the background.

3.2 Data Flow

GET Operation

1. The client sends a GET request to the controller to identify the primary worker responsible for the key.
2. Using the mapping information, the client directly contacts the primary worker.
3. The worker returns the stored value if present.

PUT Operation

1. The client sends the key-value pair to the primary worker for that key.
2. The primary worker writes the value locally.
3. The primary worker then synchronously replicates the value to the first replica.
4. An asynchronous background task replicates the value to the second replica.
5. The PUT operation is considered successful once the synchronous replication completes.

3.3 Failure Handling

Fault tolerance is supported through periodic heartbeat messages from worker nodes to the controller. If a worker fails to respond within the defined timeout period of 10 seconds, it is marked unavailable. Since the controller is assumed to be always operational, it initiates corrective actions to restore the replication factor of affected keys.

For each impacted key, the controller selects a new worker to host the missing replica and triggers replication from surviving replicas. This ensures that the system maintains three copies of each key even in the event of worker failure.

3.4 API Endpoints

Client Controller (/v1/client)

- **POST** /v1/client/put – Store a key-value pair in the distributed system.
- **GET** /v1/client/get – Retrieve the primary worker responsible for a key.
- **GET** /v1/client/get/val – Retrieve the stored value from a worker.

Controller (/v1/controller)

- **POST** /v1/controller/register – Register a worker in the system.
- **POST** /v1/controller/heartbeat – Receive heartbeat signals from workers.
- **GET** /v1/controller/key-mapping/{key} – Get primary and replica workers for a key.

- **GET** `/v1/controller/workers` – List all workers and their status.
- **POST** `/v1/controller/replica/ack` – Acknowledge asynchronous replication completion.

Worker Controller (`/v1/worker`)

- **GET** `/v1/worker/status` – Check if a worker is running.
- **GET** `/v1/worker/get` – Retrieve a value stored locally.
- **POST** `/v1/worker/put` – Store a key-value pair at a primary worker.
- **PUT** `/v1/worker/replicate/{key}` – Store replicated data from another worker.
- **GET** `/v1/worker/health` – Worker health check endpoint.

3.5 Project Flow

3.5.1 System Initialization

1. Controller startup:
 - (a) Starts on port 8085
 - (b) Initializes empty worker registry
 - (c) Starts heartbeat checker (runs every 5 seconds)
 - (d) Initializes key directory for tracking key-to-worker mappings
2. Worker startup:
 - (a) Each worker starts on designated port (8081, 8082, 8083, 8084)
 - (b) Registers with controller via `/v1/controller/register`
 - (c) Begins sending periodic heartbeats to controller

3.5.2 PUT Operation Flow

Client → Controller → Primary Worker → Replicas

Client Request:

- Client calls **POST** `/v1/client/put` with the key-value pair.

Worker Selection:

- Controller receives the PUT request.
- Controller randomly selects a worker as the *primary worker*.
- Controller forwards the request to the selected primary worker.

Primary Write:

- The primary worker stores the key-value pair in its local storage.

Replication Phase:

Synchronous Replication:

- Primary worker sends the data to a second randomly selected worker.
- Waits for confirmation before proceeding.

Asynchronous Replication:

- Primary worker sends the data to a third worker in the background without waiting for acknowledgment.

Response to Client:

- Once the primary worker and the synchronous replica confirm the write, the controller returns a success response to the client.

3.5.3 Read Operation Flow (GET)

Client → Controller → Client → Worker

Client Request:

- The client sends a request to retrieve the value associated with a specific key.

Route Discovery:

- The controller is contacted to determine which worker is responsible for the requested key.
- The controller responds with the primary worker information.

Worker Information Returned:

- The client receives the necessary connection details for the primary worker.

Value Retrieval:

- The client contacts the primary worker using the received information.
- The worker searches its local storage for the requested key and returns the stored value.
- The client receives the response containing the key and its corresponding value.

3.5.4 Heartbeat and Failure Detection

Workers → Controller (Continuous)

Heartbeat Transmission:

- Each worker periodically sends a heartbeat signal to the controller.
- The controller updates the worker's last recorded heartbeat time and marks the worker as active.

Failure Detection:

- The controller runs a background monitoring task at regular intervals.
- The controller checks the time elapsed since the last received heartbeat for each worker.
- If a worker has not sent a heartbeat within the defined timeout period, it is marked as failed.

- Once detected, the controller initiates re-replication to maintain the required number of data copies.

Automatic Recovery:

- A failed worker can rejoin the system by registering again.
- Once heartbeats resume, the worker is marked active and reintegrated into the system.

3.5.5 Re-Replication Upon Worker Failure

Controller → Surviving Workers

Failure Detected:

- The controller detects a worker failure based on missing heartbeat signals.

Impact Analysis:

- The controller checks the key directory to determine which keys were stored on the failed worker.
- These keys may have been stored either as primary copies or as replicas.

Data Recovery:

- For each affected key, the controller locates another worker that still holds a valid copy.
- A new alive worker is selected to replace the failed one.
- The key-value pair is transferred from the source worker to the new target worker.

Mapping Update:

- The controller updates the internal directory to replace the failed worker with the new one.
- Primary and replica responsibilities are reassigned as needed.
- The replication factor is restored and maintained.

3.6 System Flow Diagram

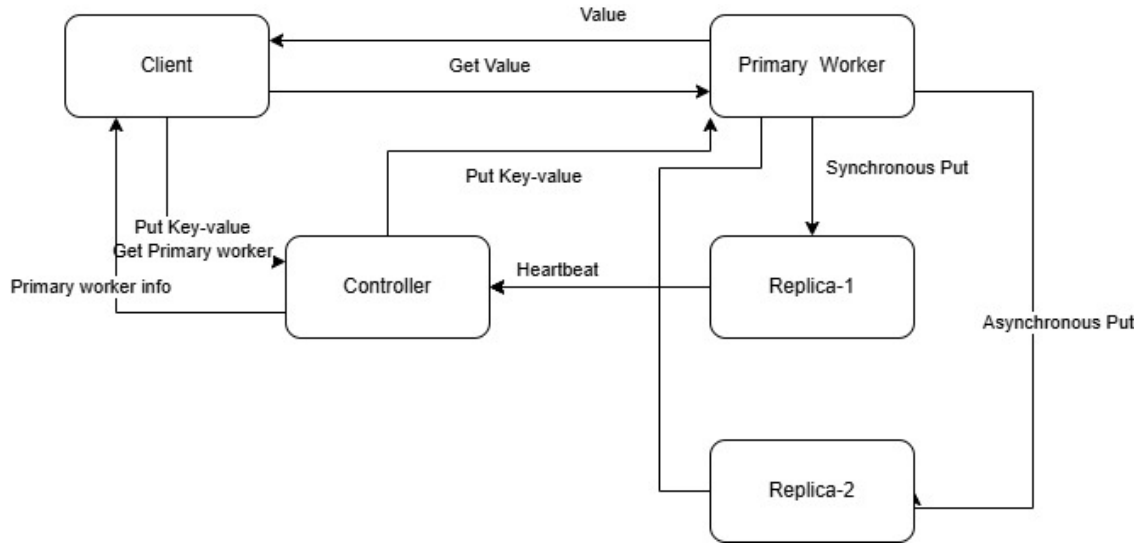


Figure 1: Put and get operation flow diagram

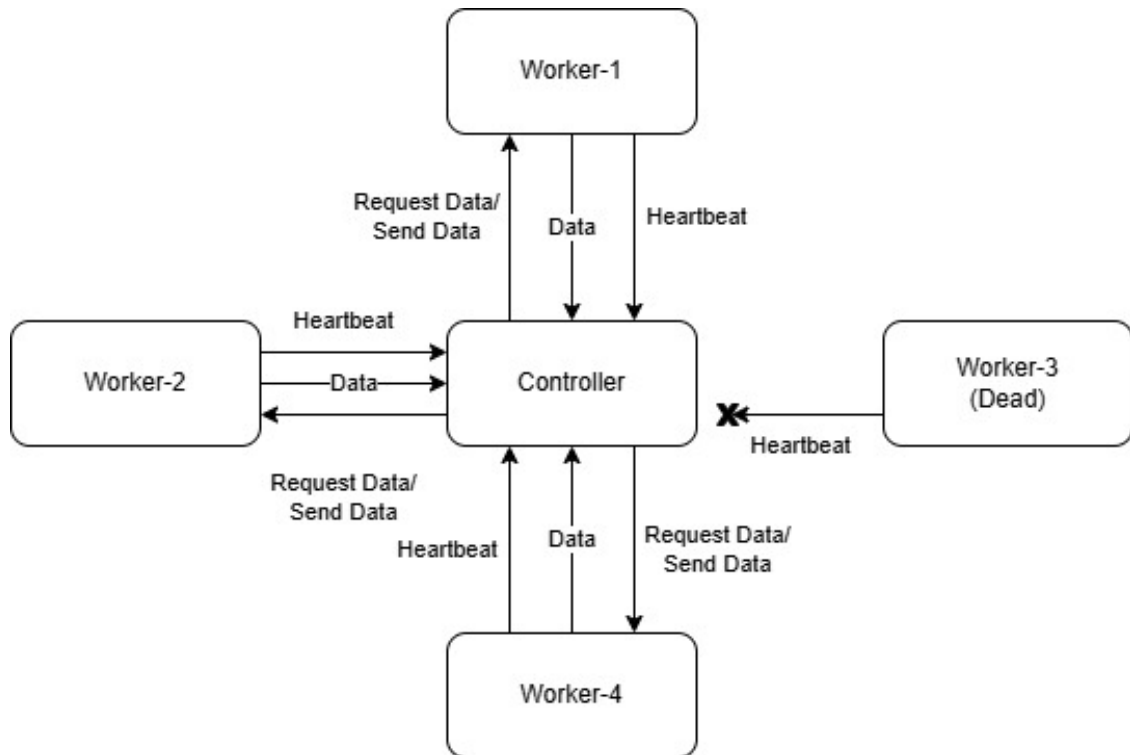


Figure 2: Re-replication flow diagram

4 AI Usage

1. The primary AI tool used by us throughout this project was ChatGPT.

2. This AI was used for the following things:
 - (a) Resolution of errors encountered during the testing of the project.
 - (b) Creation of the corresponding front-end given the backend.
3. The logic, backend, testing and cleanup of front-end was done by us.
4. The tool was exceptionally good at understanding the problem at hand and handling it to give us an error corrected code.
5. The tool did make several assumptions during the creation of the front-end such as using incorrect URLs, and since the code was very large and the problem so small it took quite a while to fix it completely.

5 Contribution

- The logic was worked on by both members.
- The Controller code was handled by Aaditya Joshi.
- The code for Client and Worker was tackled by Vaibhav Mittal.
- The process of debugging and work on report was completed in tandem.